

# Final Project – CI/CD Pipeline with AKS & ACR

Cloud Computing & DevOps Engineering

*CloudScale Kubernetes Deployment*

## Authors

| Name              | Student ID |
|-------------------|------------|
| Ayoub Elhaddad    | 5179       |
| Mohamed Aldarbag  | 4955       |
| Abdullah Fahmi    | 4827       |
| Mohamed Bin Hilal | 4957       |
| Anas Gorman       | 5038       |

## Project Title & Description

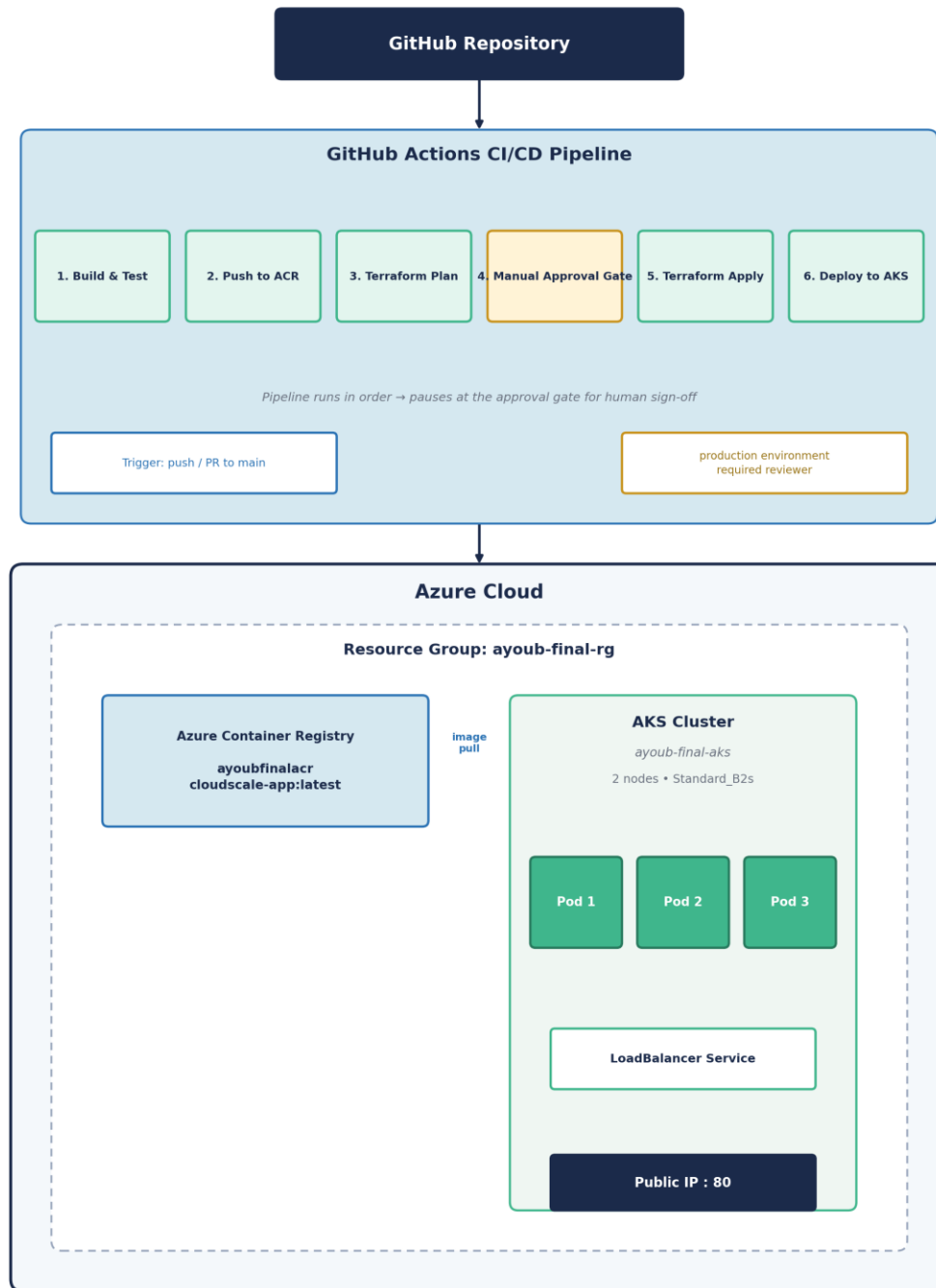
---

CloudScale Kubernetes Deployment

- Builds and tests a Dockerized Node.js web application
- Stores the image in Azure Container Registry (ACR)
- Provisions AKS + ACR infrastructure with Terraform
- Deploys the app to Azure Kubernetes Service (AKS) with 3 replicas
- Automates everything via GitHub Actions with a manual approval gate

## Architecture Diagram

---



## Setup Instructions

---

**Prerequisites:** Docker Desktop, Terraform  $\geq 1.5$ , Azure CLI, kubectl

## 1. Docker – Build & Test Locally

```
docker build -t cloudscape-app:latest .
docker run -p 3000:3000 cloudscape-app:latest
# open http://localhost:3000 | health: http://localhost:3000/health
```

## 2. Terraform – Provision Infrastructure

```
cd terraform
terraform init
terraform plan
terraform apply
```

After apply, note outputs: `acr_login_server`, `aks_cluster_name`, `resource_group_name` — add these as GitHub Secrets.

## 3. Kubernetes – Connect kubectl

```
az aks get-credentials --resource-group ayoub-final-rg --name ayoub-final-aks
kubectl get nodes
```

## 4. Push Image to ACR (first time)

```
az acr login --name ayoubfinalacr
docker build -t ayoubfinalacr.azurecr.io/cloudscale-app:latest .
docker push ayoubfinalacr.azurecr.io/cloudscale-app:latest
```

## GitHub Actions Workflow Explanation

| Job             | Trigger         | What it does                                         |
|-----------------|-----------------|------------------------------------------------------|
| build-and-test  | Every push & PR | Builds Docker image, runs /health check test         |
| push-to-acr     | Push to main    | Logs into ACR, pushes image with commit SHA tag      |
| terraform-plan  | Push to main    | Runs init, plan, saves plan artifact                 |
| manual-approval | After plan      | Pauses for human approval via production environment |
| terraform-apply | After approval  | Applies saved Terraform plan                         |
| deploy-to-aks   | After apply     | Updates manifest with new image, applies to AKS      |

## GitHub Secrets Required

| Secret              | Description                 |
|---------------------|-----------------------------|
| ARM_CLIENT_ID       | Service Principal client ID |
| ARM_CLIENT_SECRET   | Service Principal secret    |
| ARM_SUBSCRIPTION_ID | Azure subscription ID       |

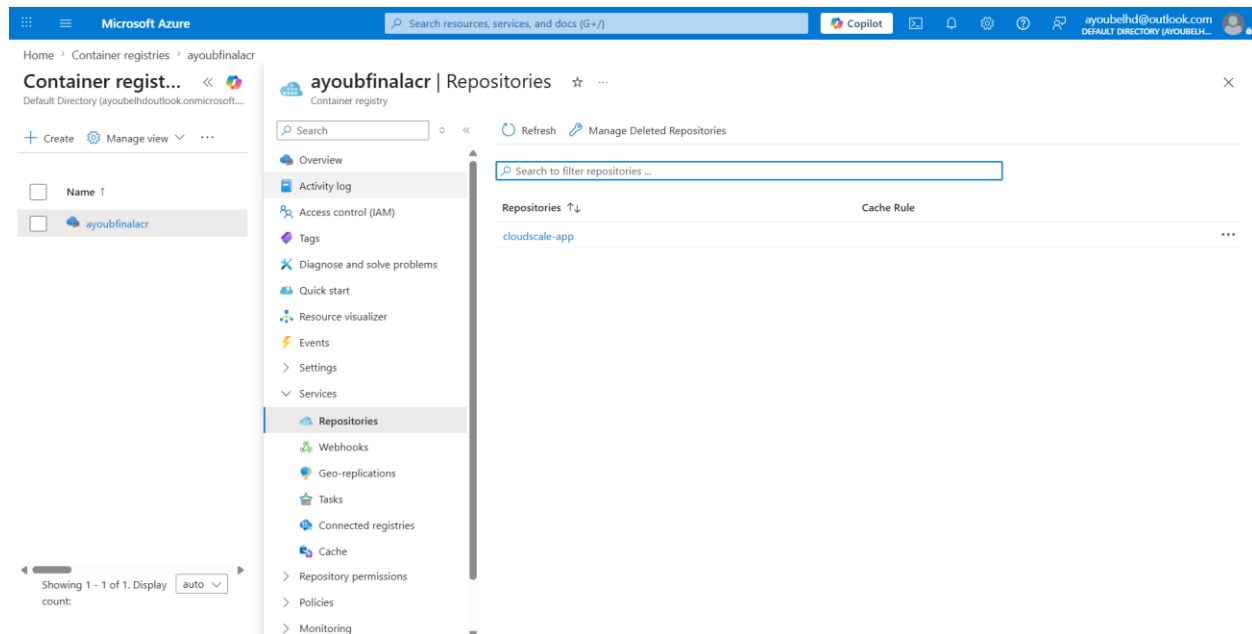
|                     |                                     |
|---------------------|-------------------------------------|
| ARM_TENANT_ID       | Azure tenant ID                     |
| AZURE_CREDENTIALS   | Full SP JSON for azure/login action |
| ACR_LOGIN_SERVER    | e.g. ayoubfinalacr.azurecr.io       |
| ACR_NAME            | e.g. ayoubfinalacr                  |
| AKS_CLUSTER_NAME    | e.g. ayoub-final-aks                |
| RESOURCE_GROUP_NAME | e.g. ayoub-final-rg                 |

## Screenshots

### 1. Docker Build Successful

```
PS C:\Users\AGG\Desktop\final-project> docker build -t ayoubfinalacr.azurecr.io/cloudscale-app:latest .
[+] Building 8.0s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 561B                             0.1s
=> [internal] load metadata for docker.io/library/node:18-alpine 2.7s
=> [auth] library/node:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                0.1s
=> => transferring context: 2B                                    0.0s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c 0.1s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c 0.1s
=> [internal] load build context                                0.1s
=> => transferring context: 3.62kB                                0.0s
=> CACHED [2/5] WORKDIR /app                                    0.0s
=> [3/5] COPY package.json .                                    0.1s
=> [4/5] RUN npm install --omit=dev                             3.8s
=> [5/5] COPY app.js .                                          0.1s
=> exporting to image                                           0.7s
=> => exporting layers                                           0.2s
=> => exporting manifest sha256:3abd234466e4ec51bb1d0293c20cf9289d8a812d115077e35137036b26108c18 0.0s
=> => exporting config sha256:b10a907812061bc1460b8ff18d4f29c9b1097d40bfff3436c0c986658b4d75cb3 0.0s
=> => exporting attestation manifest sha256:17f61ee0ffaa69d5f39ba1eeb5121a884ef976d4e12cf1b65c52f3249ea19d7c 0.0s
=> => exporting manifest list sha256:848deb18efe99a23de56eb1ae9a6d4f3efdae26160ece4a3e9029fa0dd4ded10 0.0s
=> => naming to ayoubfinalacr.azurecr.io/cloudscale-app:latest 0.0s
=> => unpacking to ayoubfinalacr.azurecr.io/cloudscale-app:latest 0.2s
```

### 2. Image in ACR (Azure Container Registry)



### 3. Terraform Apply Successful

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

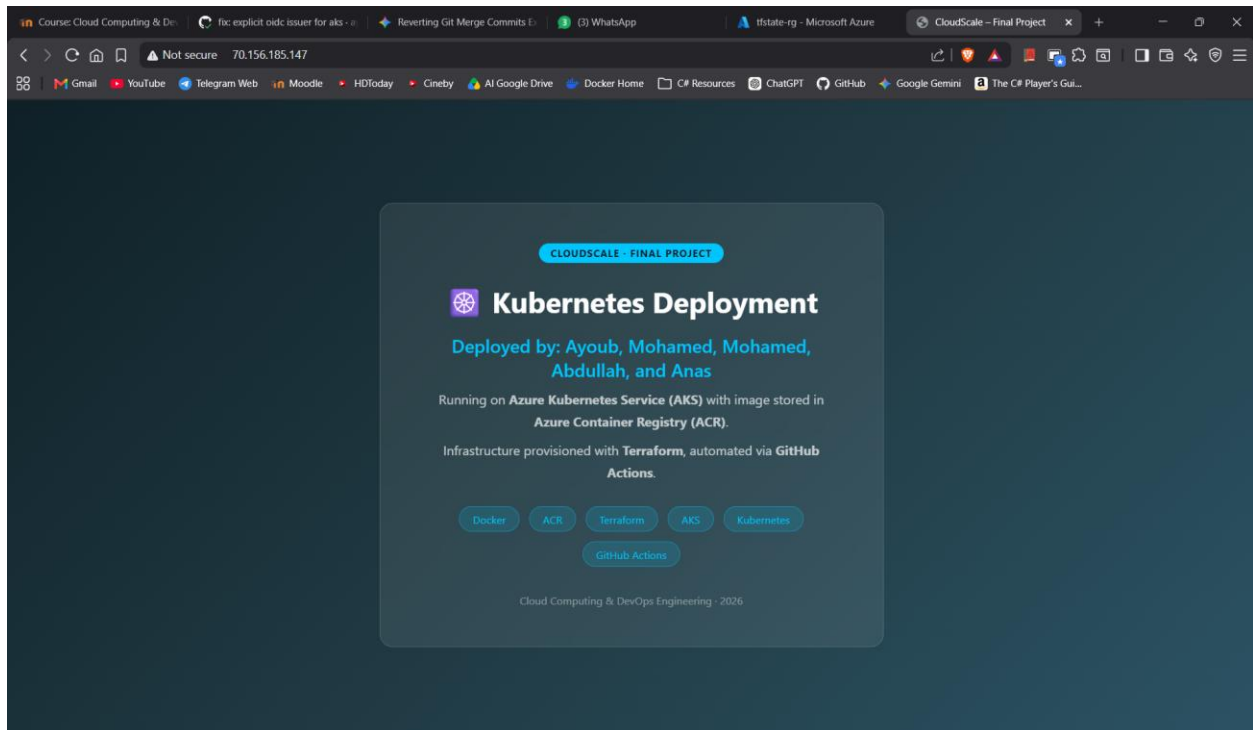
Outputs:

```
acr_login_server = "ayoubfinalacr.azurecr.io"
acr_name = "ayoubfinalacr"
aks_cluster_name = "ayoub-final-aks"
aks_kube_config = <sensitive>
resource_group_name = "ayoub-final-rg"
```

### 4. AKS Nodes Ready (kubectl get nodes)

```
PS C:\Users\AGG\Desktop\final-project> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
aks-default-21290007-vmss000000    Ready     <none>    14m   v1.34.8
aks-default-21290007-vmss000001    Ready     <none>    14m   v1.34.8
PS C:\Users\AGG\Desktop\final-project> |
```

### 5. Application Running in Browser



## 6. GitHub Actions Workflow Successful

The screenshot shows the GitHub Actions interface for a workflow named "chore: test pr trigger #5". The workflow is in a "Success" state, triggered via a pull request 1 minute ago. The total duration is 27s. The workflow file is "ci-cd.yml" on the "pull\_request" event. The workflow steps are: Build and Test Docker Image (24s), Push Image to ACR, Terraform Plan, Manual Approval, Terraform Apply, and Deploy to AKS. The left sidebar shows the workflow steps and run details. The bottom section shows annotations, including 1 warning.

## 7. GitHub Actions Approval Gate

The screenshot shows a GitHub pull request titled "chore: test pr trigger #1" by user "ayaelhd". The pull request is open, showing 1 commit, 6 checks, and 1 file changed. A comment by "ayaelhd" states "No description provided." Below the comment, a status bar indicates "chore: test pr trigger" with a green checkmark and the commit hash "9212cc0". A message states "This branch has not been deployed" with "No deployments". A green box indicates "All checks have passed" (5 skipped, 1 successful checks) and "No conflicts with base branch" (Merging can be performed automatically). A green button labeled "Merge pull request" is visible. At the bottom, it says "Still in progress? Convert to draft".

## 8. Azure Portal Showing AKS + ACR

ayoub-final-rg  
Resource group

Search

Create Manage view Delete resource group Refresh Group by none

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management
- Monitoring
- Automation
- Help

Essentials

Resources Recommendations

Filter for any field... Type equals all Location equals all Add filter

| Name ↑          | Type               | Location      |
|-----------------|--------------------|---------------|
| ayoub-final-aks | Kubernetes service | Spain Central |
| ayoubfinalacr   | Container registry | Spain Central |

## Step-by-Step Solution

1. Write app.js with /health endpoint and HTML page displaying team names
2. Write Dockerfile using node:18-alpine, copy app, expose port 3000, add HEALTHCHECK
3. Write Terraform files: providers.tf (AzureRM + remote backend), variables.tf (5 variables), main.tf (Resource Group + ACR + AKS + role assignment), outputs.tf (5 outputs)
4. Grant AKS the AcrPull role via azure\_rm\_role\_assignment so it pulls from ACR without secrets
5. Write k8s/deployment.yaml with 3 replicas, readiness probe and liveness probe on /health
6. Write k8s/service.yaml as LoadBalancer type mapping port 80 to container port 3000
7. Write GitHub Actions ci-cd.yml with 6 jobs: build-and-test, push-to-acr, terraform-plan, manual-approval, terraform-apply, deploy-to-aks
8. Run terraform apply locally to provision Resource Group, ACR, and AKS cluster
9. Push Docker image to ACR manually for the first time
10. Connect kubectl with az aks get-credentials and verify nodes with kubectl get nodes
11. Add all 9 GitHub Secrets to repo settings
12. Create production environment in GitHub with required reviewer for manual approval gate

13. Push code to feature branch, open Pull Request, verify build-and-test job passes
14. Merge PR, approve deployment in GitHub Actions, verify all 6 jobs complete successfully
15. Get external IP from kubectl get service and verify app loads in browser
16. Run terraform destroy to save Azure credits after taking all screenshots

## Repository Link

---

<https://github.com/ayaelhd/cloudscale-final-project>

---

**Cost Reminder:** Run terraform destroy after completing the project. AKS costs approximately \$1.60/day.